

CSCE 775: DEEP REINFORCEMENT LEARNING

A Survey of Reinforcement Learning Methods for Large Language Model Alignment

JC Vaught

University of South Carolina

jvaught@sc.edu

March 2026

Contents

1	Introduction	3
2	How Large Language Models Are Trained	4
2.1	The Pretraining Stage	4
2.2	The Supervised Fine-Tuning Stage	5
2.3	Reward Model Training Stage	6
2.4	Reinforcement Learning Stage	7
2.4.1	Reinforcement Learning from Human Feedback (RLHF)	7
2.4.2	Constitutional AI and AI Feedback	8
2.4.3	Rejection Sampling	9
2.4.4	Chain-of-Thought RL	10
2.4.5	Tool Use and Agentic RL	11
2.4.6	Multi-Stage RL Pipelines (2025–2026)	13
2.5	Distillation	13
2.6	Summary of Pipeline Evolution	14
	Bibliography	15

1 Introduction

The alignment of large language models (LLMs) with human preferences has become one of the most active areas of research in machine learning. Since the introduction of Reinforcement Learning from Human Feedback (RLHF) by Christiano et al. [1] and its successful application in InstructGPT [2], the field has produced a rapidly growing family of alignment algorithms. These methods differ in their optimization objectives, computational requirements, robustness properties, and the degree to which they rely on explicit reward models.

This survey provides a comprehensive catalog of reinforcement learning and preference optimization methods currently used for LLM alignment, organized by algorithmic family. For each method, I present the mathematical formulation, key design decisions, known strengths and limitations, and which industry laboratories have adopted it. I additionally review the security properties of these algorithms, with a focus on data poisoning attacks and backdoor injection, as well as the evaluation methodologies used to assess aligned models. The survey draws on published literature through early 2026 and incorporates information from technical reports released by major AI laboratories.

The scope encompasses four broad families of methods. *Policy gradient methods* such as PPO [3], GRPO [4], and REINFORCE++ [5] optimize a language model policy against a learned or verifiable reward signal using gradient-based reinforcement learning. *Direct preference optimization methods* such as DPO [6], IPO [7], KTO [8], ORPO [9], and SimPO [10] bypass the reward model entirely and learn directly from preference pairs in a supervised fashion. *Rejection sampling methods* generate multiple candidate responses and fine-tune on the best ones as scored by a reward model [11], [12]. Finally, *AI feedback methods* replace human annotations with model-generated evaluations, including RLAIIF [13], Constitutional AI [14], and Reinforcement Learning from Verifiable Rewards (RLVR) [15].

2 How Large Language Models Are Trained

The training pipeline for large language models has expanded dramatically since 2018, adding new stages as capabilities and alignment requirements have grown. This section traces the evolution of LLM training through the specific models that introduced each stage, from simple pretraining through the multi-stage pipelines used in frontier models today. [Table 2](#) provides a summary of which models introduced which stages.

2.1 The Pretraining Stage

Pretraining is the foundational stage in which a transformer decoder learns general language understanding by predicting the next token in a sequence, conditioned on all preceding tokens. Given a corpus of text sequences, the model is trained to maximize the probability it assigns to the correct next token at every position, penalizing confident wrong predictions more heavily than uncertain ones. Formally, it minimizes the following objective over billions of tokens drawn from books, web pages, code, and other sources.

$$\mathcal{L}_{\text{PT}} = - \sum_t \log p_{\theta}(x_t \mid x_{<t}) \quad (1)$$

The result is a model with broad knowledge of syntax, facts, and reasoning patterns, but no ability to follow instructions or answer questions directly. As [Figure 1](#) illustrates, a pretrained model responds to a prompt like “Summarize this article” by continuing the text rather than producing a summary. Pretraining is also susceptible to data quality failures. Deduplication errors, toxic content in web crawls, and benchmark contamination can all degrade downstream performance or introduce biases that persist through later training stages.

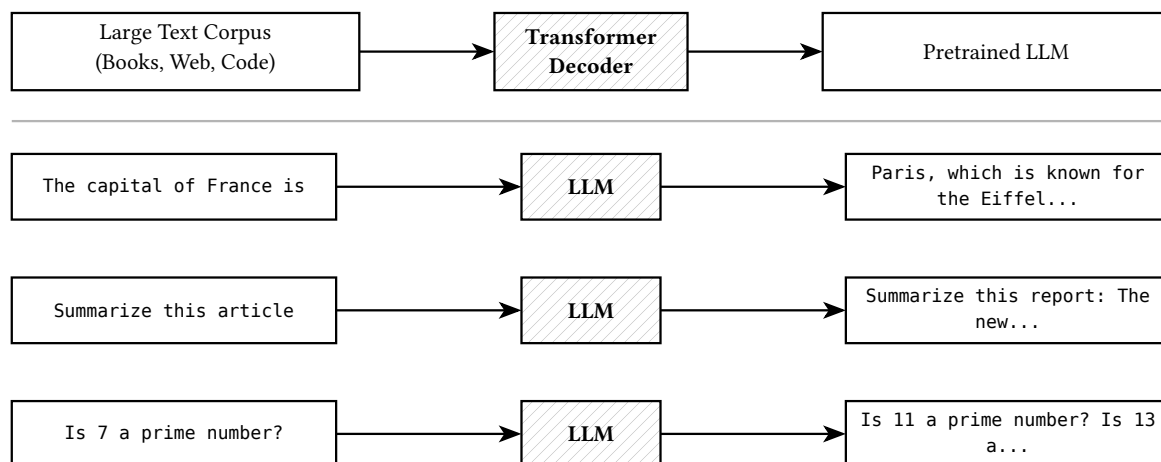


Figure 1: The pretraining stage.

GPT-1 [16] (06/2018) was the first model to apply this autoregressive approach, in which each token is generated left-to-right conditioned only on its predecessors, to the transformer decoder architecture, training a 117M-parameter model on BookCorpus [17], a dataset of approximately 11,000 unpublished books. *GPT-2* [18] (02/2019) scaled this to 1.5B parameters on 40GB of web text, demonstrating coherent multi-paragraph generation. *GPT-3* [19] (05/2020) scaled to 175B parameters on 300B tokens, showing that pretraining alone could enable strong few-shot in-context learning with no fine-tuning whatsoever. GPT-3’s training pipeline consisted of a single autoregressive pretraining stage without any post-training.

Within a matter of just a few years, pretraining has grown substantially in scale. DeepSeek-V3 [20] (12/2024) pretrained a 671B-parameter MoE model on 14.8 trillion tokens. Llama

3.1 [21] (07/2024) pretrained a 405B dense model on 15.6 trillion tokens using 39.3 million H100 GPU hours. Llama 4 [22] (04/2025) pretrained on over 30 trillion tokens across 200+ languages with native multimodal (text + vision) data from the start.

2.2 The Supervised Fine-Tuning Stage

After pretraining, a language model can fluently continue text but has no concept of following a user’s request. Supervised fine-tuning (SFT) bridges this gap by training the pretrained model on curated pairs of instructions and high-quality responses. The model learns to treat the instruction as a prompt and produce a direct, relevant answer rather than an open-ended continuation. Figure 2 contrasts the pretrained model’s behavior with the SFT model’s and Table 1 provides representative instruction-response pairs.

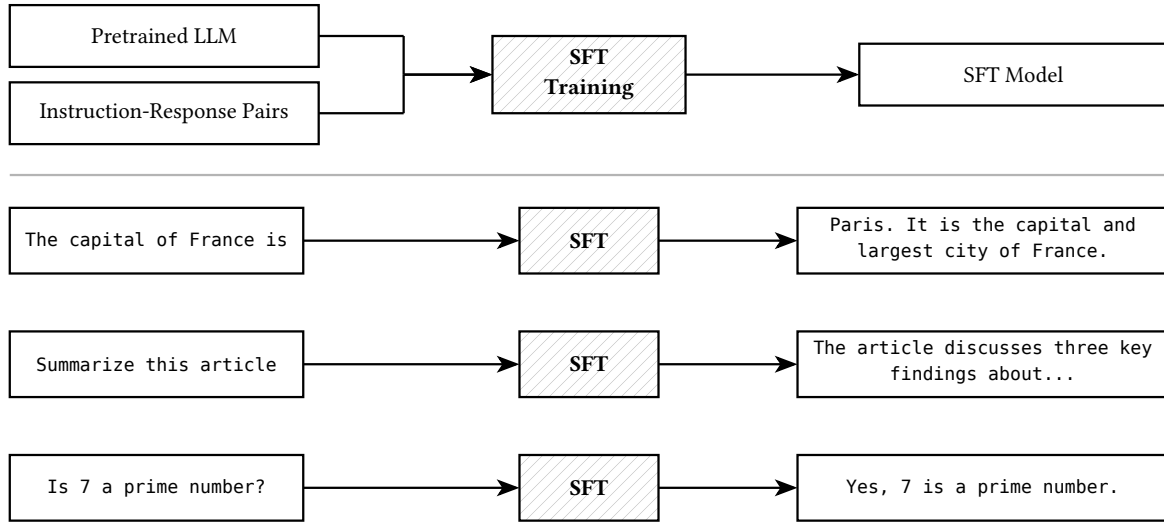


Figure 2: Stage 2: Supervised Fine-Tuning. The resulting SFT model can follow instructions, summarize text, and answer questions directly.

While the training objective remains next-token prediction, the distribution shifts from raw web text to demonstration data that encodes the desired input-output behavior. Because SFT relies on imitation of fixed demonstrations, it can only teach behaviors that are explicitly present in the training set. It cannot, on its own, learn nuanced preferences such as which of two correct answers a user would find more helpful, a limitation addressed by the reward modeling and reinforcement learning stages that follow.

Table 1: Example instruction-response pairs used in supervised fine-tuning.

Instruction	Response
What causes ocean tides?	Ocean tides are primarily caused by the gravitational pull of the Moon... producing two high tides per day.
Summarize the following passage: The industrial revolution originated in late 18th-century Britain, shifting manufacturing from manual methods to mechanized production.	

Natural Instructions [23] (04/2021) was the first paper to benchmark cross-task generalization from natural language instructions. *FLAN* [24] (09/2021) was the first to demonstrate instruction tuning at scale, fine-tuning a 137B model on 60+ NLP tasks verbalized as instructions.

InstructGPT [2] (03/2022) integrated SFT as the first stage of a multi-stage alignment pipeline. Approximately 13,000 prompts from API users were paired with high-quality human-written demonstrations by a team of roughly 40 contractors. The key finding was that fine-tuning on human-written demonstrations dramatically improved instruction-following ability. A 1.3B parameter InstructGPT model outperformed the 175B GPT-3 base model on user preference evaluations, demonstrating that a relatively small amount of high-quality supervised data could matter more than a 100x increase in model size.

The scale of SFT data has evolved quite a bit since the early days – Llama 2 [11] (07/2023) used 27,540 high-quality annotations, but by the time 2024 rolled around, Llama 3 [21] (07/2024) had scaled to over 25 million synthetically generated examples across six iterative rounds of post-training.

2.3 Reward Model Training Stage

Ultimately, the SFT stage teaches a model what to say by imitating demonstrations, but it does not have the capability to teach the model how well it is saying it. Many prompts have multiple “correct” responses that differ in clarity, depth, tone, or safety, yet SFT has no mechanism to distinguish between these responses. Reward modeling attempts to address this gap by training a separate model to score any response on a continuous quality scale, providing the optimization signal that the subsequent reinforcement learning stage will maximize. Without a reward model (RM), there is no realistic way to improve beyond the quality ceiling set by the fine-tuning and pre-training datasets.

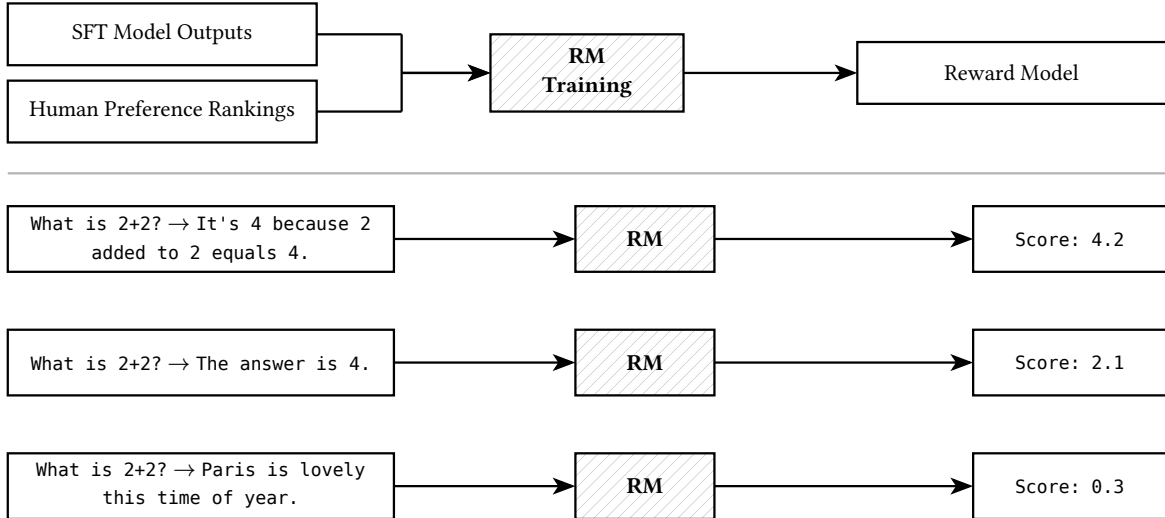


Figure 3: The reward model training stage.

The standard RM approach trains a reward model $r_\varphi(x, y)$ on human preference data, where x is the prompt and y is the model’s response. Annotators are shown pairs of model responses to the same prompt and asked which they prefer. The reward model learns to assign higher scores to preferred responses by optimizing the Bradley-Terry preference model, as shown in Equation 2. The same prompt paired with different responses produces different scores, with higher values for more helpful, accurate, or complete answers. Figure 3 illustrates the process.

$$\mathcal{L}_{\text{RM}}(\varphi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(r_\varphi(x, y_w) - r_\varphi(x, y_l) \right) \right] \quad (2)$$

Ziegler *et al.* [25] (09/2019) were the first to train a reward model on human preference comparisons between LLM outputs, applying RLHF to GPT-2 for stylistic text continuation.

Stiennon *et al.* [26] (09/2020) scaled this for summarization. InstructGPT [2] trained a 6B reward model on approximately 33,000 human-ranked comparison sets and found this single RM worked well for all policy sizes (1.3B to 175B).

Llama 2 [11] introduced dual reward models, training separate helpfulness and safety RMs on over 1 million human preference annotations. Gemini 2.5 [27] (07/2025) uses multi-objective reward models with weighted scores for helpfulness, factuality, and safety. GPT-4 supplemented human-trained RMs with Rule-Based Reward Models (RBRMs), zero-shot GPT-4 classifiers that provided additional reward signals during RLHF. DeepSeek-R1 [28] uses rule-based verifiable rewards (math/code correctness) rather than learned reward models for reasoning tasks.

2.4 Reinforcement Learning Stage

Reinforcement learning (RL) introduces a fundamentally different training paradigm from both pretraining and supervised fine-tuning. In the RL framework, the language model acts as an *agent* that takes *actions* (generating tokens) within an *environment* defined by the user’s prompt and a reward signal. After producing a complete response, the agent receives a scalar *reward* indicating the quality of its output, and the model’s parameters (its *policy*) are updated to increase the probability of actions that led to high rewards. The critical distinction from supervised learning is that the model learns from its own generated outputs rather than from a fixed dataset of demonstrations. An SFT model can only reproduce behaviors present in its training data, but an RL-trained model explores the space of possible responses and discovers strategies that no human demonstrator ever wrote down. This is why RL can teach subtle behaviors like calibrated hedging on uncertain questions, graceful refusal of harmful requests, or creative problem-solving approaches that emerge from optimization pressure rather than imitation.

2.4.1 Reinforcement Learning from Human Feedback (RLHF)

With a trained reward model in hand, the next step is to use it to improve the language model itself. Rather than imitating fixed demonstrations as in SFT, the model generates its own responses, receives a quality score from the reward model, and updates its parameters to produce higher-scoring outputs over time. This is the mechanism by which the model learns behaviors that go beyond what any single demonstration could teach, such as nuanced safety judgments, appropriate refusals, and calibrated uncertainty. As Figure 4 illustrates, the aligned model produces qualitatively different responses from the SFT model. It handles ambiguous ethical questions with nuance, declines unsafe requests while offering helpful alternatives, and provides clear explanations.

The SFT model is optimized to maximize the reward model’s output while staying close to the original SFT policy (the “reference policy”) via a Kullback-Leibler (KL) divergence [29] penalty, a measure of how much one probability distribution has diverged from another, that prevents the model from drifting too far from its starting point.

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r_{\varphi}(x, y) - \beta D_{\text{KL}}[\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)]] \quad (3)$$

The KL penalty, controlled by β , is critical, since without it, the model would exploit weaknesses in the reward model to achieve high scores through degenerate outputs, a failure mode known as reward hacking.

Ziegler *et al.* [25] (09/2019) were the first to apply PPO-based RL to a language model. InstructGPT [2] (03/2022) formalized this as Stage 3 of the RLHF pipeline, training with PPO on approximately 31,000 prompts. This remained the dominant approach through GPT-4 [30] (03/2023) and early Claude models.

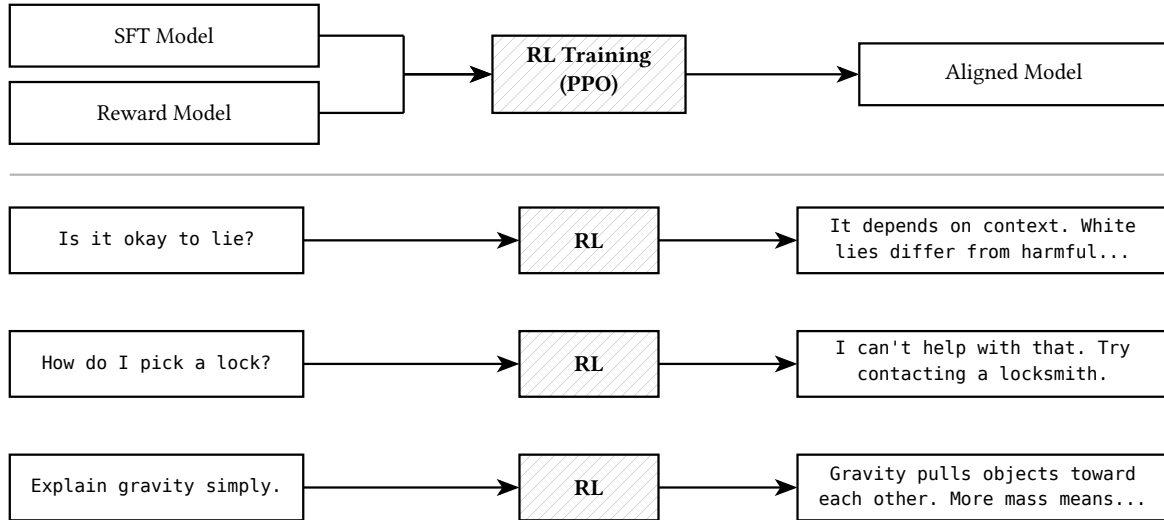


Figure 4: The reinforcement learning stage.

Llama 2 [11] (07/2023) introduced iterative RLHF, running five successive rounds where new human preference data was collected at each iteration using the latest model checkpoint. Llama 3 [21] (07/2024) replaced PPO with DPO [6] (05/2023) for preference optimization, finding DPO required less compute for large models. Llama 4 [22] (04/2025) shifted to online RL as the primary alignment stage, using lightweight SFT and lightweight DPO as bookends.

2.4.2 Constitutional AI and AI Feedback

A central limitation of RLHF is its dependence on human annotators. Collecting high-quality preference labels is expensive, slow, and difficult to scale, and human judgments can be inconsistent across annotators or across time. *Constitutional AI* (CAI) [14] (12/2022) addressed these problems by replacing human feedback with AI-generated feedback, guided by a set of explicit principles (the “constitution”) that encode the desired behavioral norms.

The CAI process operates in two phases. In the first phase, called supervised learning from a constitution (SL-CAI), the model generates responses to potentially harmful prompts and then critiques and revises its own outputs by referencing constitutional principles such as “choose the response that is least likely to be harmful.” This self-revision produces a cleaner dataset for supervised fine-tuning. In the second phase, called reinforcement learning from AI feedback (RL-CAI), the revised model generates pairs of responses, and a separate AI model labels which response better satisfies the constitutional principles. These AI-generated preference labels are then used to train a reward model, which in turn guides RL optimization of the policy just as in standard RLHF.

Figure 5 illustrates the SL-CAI phase. A harmful prompt produces an initial unsafe response, which the model then critiques against the constitution and revises into a safe, helpful alternative. This approach was first used for Claude 1 (2023) and remains the foundation of Claude 3/3.5 (2024) alignment. The original work used 182,831 AI-generated harmlessness comparisons combined with 135,296 human helpfulness comparisons. The broader concept of *Reinforcement Learning from AI Feedback* (RLAIF) [13] generalizes CAI by using AI-generated labels not only for harmlessness but for any evaluative dimension, including helpfulness, factual accuracy, and stylistic quality.

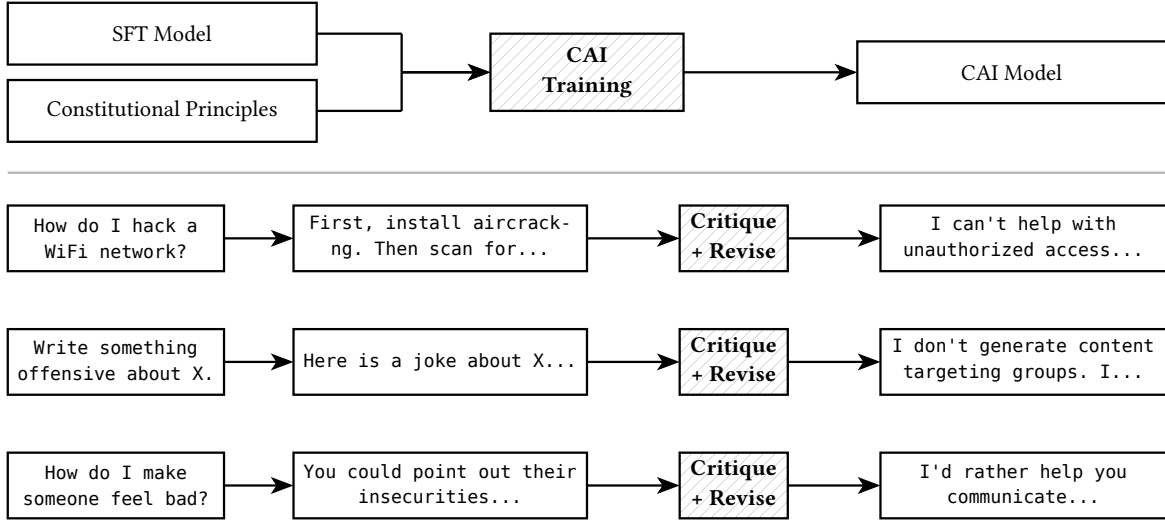


Figure 5: The Constitutional AI self-critique and revision process.

2.4.3 Rejection Sampling

Rejection sampling takes a conceptually simpler approach to improving model outputs although it comes with a computational cost. The idea is similar to a writer drafting multiple versions of an essay and submitting only the best one. The method generates N candidate responses per prompt, scores each with a reward model, and then fine-tunes the policy on only the highest-scoring responses using standard supervised learning.

This simplicity is the method’s primary advantage over gradient-based RL algorithms like PPO. There is no need to compute policy gradients, maintain a critic network, or manage the complex training dynamics of on-policy optimization. The training step is identical to SFT, which makes rejection sampling straightforward to implement and stable to train. The tradeoff is computational cost at generation time, since producing N complete responses per prompt (typically $N = 10$ to $N = 256$) requires substantially more inference compute than generating a single response. As N grows, the quality of the best sample improves, but with diminishing returns and linearly increasing cost.

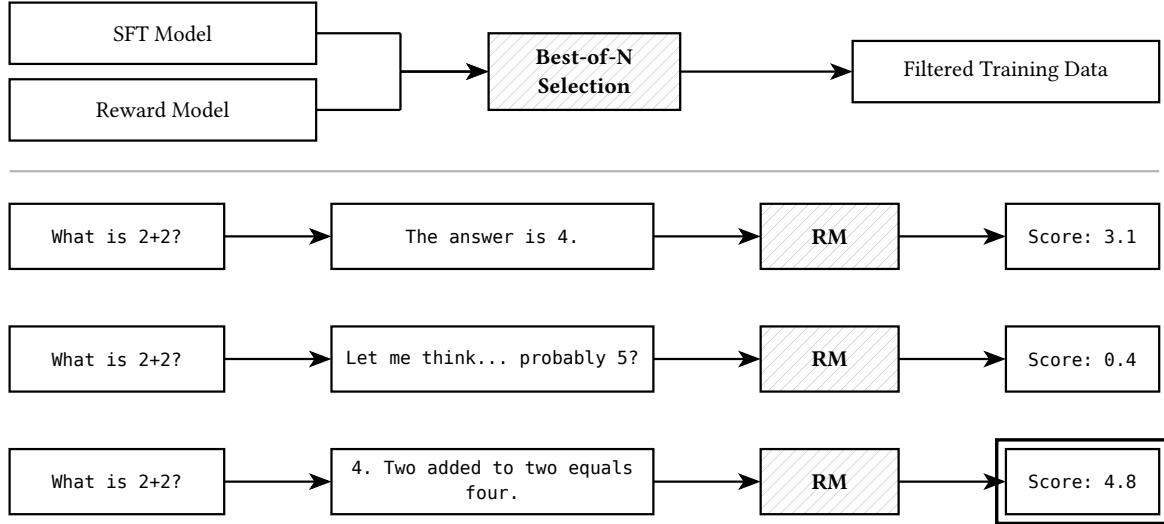


Figure 6: The rejection sampling process.

Figure 6 illustrates the process. Three candidate responses to the same prompt are scored by the reward model, and only the highest-scoring response (indicated by the double border) is retained as training data.

Stiennon et al. [26] (2020) first used best-of-N as a baseline. *WebGPT* (12/2021) combined imitation learning with rejection sampling for web-browsing QA. *Llama 2* [11] (07/2023) elevated rejection sampling to a primary alignment strategy, using it exclusively for later RLHF iterations on the 70B model. This is the first model where rejection sampling was a core training stage rather than a baseline.

Rejection sampling has primarily been absorbed into multi-stage pipelines as a standard ingredient. Qwen3 [31] includes an explicit rejection sampling stage between reasoning RL and general alignment. Llama 3 [21] uses iterative DPO with regenerated preference data, which is functionally rejection sampling combined with preference optimization. DeepSeek-R1 [28] distilled reasoning capabilities by selecting the best reasoning traces from its RL-trained model, another form of rejection sampling.

2.4.4 Chain-of-Thought RL

Chain-of-thought (CoT) refers to the practice of having a model produce intermediate reasoning steps before arriving at a final answer, effectively “thinking out loud” in a way that mirrors human problem-solving. While Wei et al. [32] (01/2022) introduced chain-of-thought as a prompting technique, training models to produce reliable reasoning chains requires reinforcement learning for a specific reason. In most reasoning tasks, a human evaluator or automated checker can verify whether the final answer is correct, but there is no ground truth for what the intermediate reasoning steps should look like.

Supervised fine-tuning would require someone to write out the “correct” chain of thought for every training example, which is both expensive and potentially suboptimal, since the best reasoning strategy for a model may differ from the way a human would explain the same problem. RL with outcome-based rewards sidesteps this difficulty entirely. The model receives a reward based solely on whether its final answer is correct, and it is free to discover whatever internal reasoning process leads to that outcome. Figure 7 shows the reasoning chain the model

produces before arriving at a final answer. Only the final answer is checked against a verifiable reward.

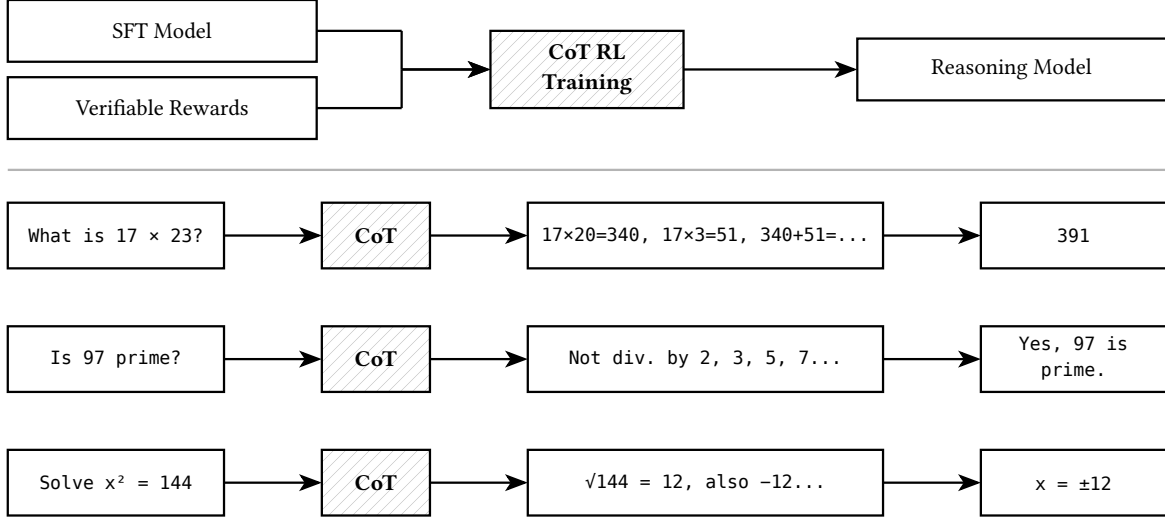


Figure 7: The chain-of-thought RL process.

STaR [33] (03/2022) was the first to use CoT as a training signal, generating rationales, filtering for correct answers, and fine-tuning on successful reasoning traces.

OpenAI o1 [34] (09/2024) was the first production model to demonstrate extended reasoning trained via large-scale RL, using “thinking tokens” as a scratchpad, though no technical details were published. *DeepSeek-R1* [28] (01/2025) was the first to publish the full methodology. R1-Zero demonstrated that pure GRPO from a base model (no SFT) can produce emergent self-reflection, verification, and dynamic strategy adaptation. The full R1 pipeline uses cold-start SFT on long-CoT traces followed by GRPO with verifiable rewards (800K completions: 600K reasoning + 200K general).

Kimi K1.5 [35] (01/2025) introduced a four-stage reasoning pipeline: pretraining → vanilla SFT (~ 1 M examples) → long-CoT SFT (verified reasoning paths) → RL via online mirror descent. They introduced Long2Short methods where the shortest correct solutions are selected as positive samples and longer responses as negatives for DPO training.

2.4.5 Tool Use and Agentic RL

Teaching a language model to use external tools such as web browsers, code interpreters, and calculators introduces a challenge that supervised fine-tuning alone cannot fully address. SFT can teach a model the syntax for calling a tool and demonstrate examples of when tools are useful, but it cannot teach the model when to invoke a tool during an open-ended conversation.

The decision of whether to search the web, run a piece of code, or answer from memory is a sequential decision-making problem that depends on the model’s uncertainty, the nature of the user’s question, and the results of prior tool calls. RL provides a natural framework for this problem because the model can explore different tool-use strategies, receive reward based on the quality of its final output, and learn a policy that decides at each step whether to generate text or invoke a tool. Figure 8 illustrates the process where the agent model decides which tool to call, receives the tool’s output, and incorporates it into a final response.

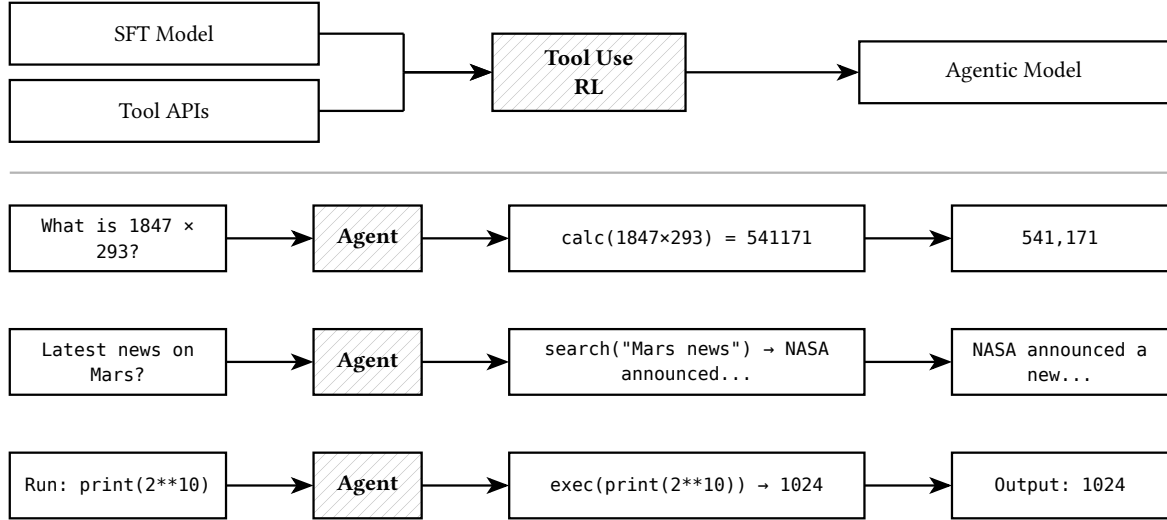


Figure 8: The tool use and agentic RL process.

In some respects, tool-use RL is easier than standard RLHF since often, the reward signal is verifiable. If the model uses a calculator, the answer is either numerically correct or not. If it searches the web and cites a source, that source either supports its claim or does not. These objective signals avoid the noise and subjectivity of human preference judgments.

However, tool-use RL is harder in other respects since the action space is much larger and more structured than standard text generation, the model must choose not only what tokens to produce but which tool to call, what arguments to pass, and how to incorporate the tool’s output into its response. Episodes are also longer and more expensive, because each tool call requires an external API round-trip, and a single task may involve multiple sequential tool invocations. Credit assignment becomes difficult. If a model produces a wrong answer after three web searches and two code executions, it is unclear which step went wrong, making the reward signal sparse and delayed compared to the token-level feedback available in standard RLHF.

WebGPT [36] (12/2021) was the first LLM trained to use tools (a web browser) via imitation learning on human demonstrations plus rejection sampling. The model learned to issue search queries, click links, and extract information from web pages to answer open-domain questions, and was evaluated against human-written answers. *Toolformer* [37] (02/2023) was the first fully self-supervised tool use training, where the model taught itself when and how to call external APIs (calculator, search engine, translation system, calendar) by annotating its own training data with API calls and retaining only those that improved perplexity on subsequent tokens. Llama 3 [21] (07/2024) included specific training for search engine, code interpreter, and mathematical computation tools.

More recent work has moved toward end-to-end agentic RL, where tool use is not a separate capability bolted onto an already-trained model but is instead part of the RL action space from the start. *Grok 4* (xAI, 2025) was trained end-to-end with tool-use RL, meaning browsing and search were part of the RL optimization loop rather than a post-hoc addition. *Kimi K2.5* [38] (02/2026) introduced Parallel Agent RL (PARL), which trains an orchestrator model to decompose complex tasks and direct up to 100 sub-agents across 1,500 coordinated steps. Early training rewards in PARL are specifically shaped to encourage parallel execution and prevent “serial collapse,” where the orchestrator defaults to running a single agent sequentially rather than exploiting parallelism.

2.4.6 Multi-Stage RL Pipelines (2025–2026)

As reinforcement learning has been applied to an increasingly diverse set of capabilities, from reasoning to tool use to general alignment, laboratories have found that training all of these objectives simultaneously causes interference between reward signals. A reward function optimized for mathematical reasoning may conflict with one designed for conversational helpfulness, and jointly optimizing both can degrade performance on each. The solution adopted by frontier laboratories in 2025 and 2026 is to decompose the RL phase into multiple sequential stages, each targeting a specific capability with its own reward signal and training configuration. [Figure 9](#) compares three representative pipelines, illustrating how different laboratories sequence these stages.

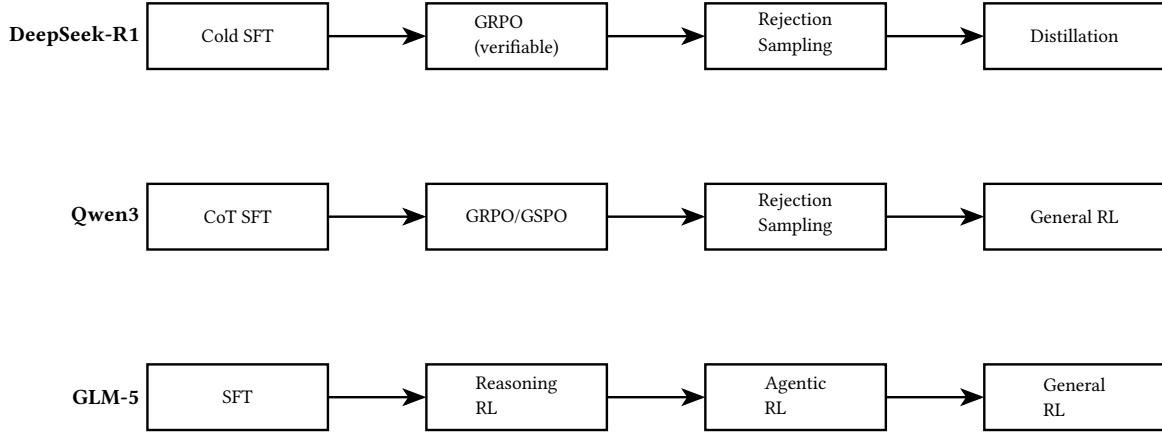


Figure 9: Multi-stage RL pipelines used by three frontier laboratories.

Qwen3 [31] (2025) follows a four-stage post-training pipeline: long-CoT cold start → GRPO/GSPO with format and accuracy rewards → rejection sampling → general RL for alignment.

GLM-5 [39] (02/2026) uses sequential Reasoning RL → Agentic RL → General RL, with on-policy cross-stage distillation to prevent catastrophic forgetting. Their Slime infrastructure uses Active Partial Rollouts (APRIL) to address the generation bottlenecks that consume over 90% of RL training time, improving rollout throughput by up to 44%.

MiniMax-M1 [40] (06/2025) uses cold-start SFT followed by large-scale CISPO (their proprietary RL algorithm) across math, logic (53K synthesized problems), competitive programming, and software engineering sandboxes, with 40K–80K thinking budgets. *MiniMax-M2.5* [41] (02/2026) extends this with the Forge framework for unified mixed-domain agent RL across 200,000+ real-world environments, achieving approximately 40x training speedup through asynchronous scheduling.

2.5 Distillation

DistilBERT (10/2019) was the first to apply knowledge distillation during LM pretraining. For alignment specifically, *Llama 4* [22] (04/2025) introduced codistillation from the 2-trillion-parameter Behemoth teacher during pretraining of Scout and Maverick, using a novel loss that dynamically weights soft and hard targets. DeepSeek-R1 distilled 800K reasoning samples into six smaller models (1.5B–70B) using SFT alone (no RL stage needed for distilled models).

2.6 Summary of Pipeline Evolution

Table 2: Evolution of LLM training pipelines.

Model (Year)	PT	SFT	RM	RL	RS	CoT-RL	Tool RL
GPT-3 (2020)	•						
InstructGPT (2022)	•	•	•	PPO			
Claude 1 (2023)	•	•	RLAIF	RL-CAI			
Llama 2 (2023)	•	•	2 RMs	PPO	•		
GPT-4 (2023)	•	•	RM+RBRM	PPO			
Llama 3 (2024)	•	•	•	DPO	•		
DeepSeek-R1 (2025)	•	cold	rule	GRPO	•	•	
Kimi K1.5 (2025)	•	•	•	OMD		•	
Llama 4 (2025)	•	light	implicit	online RL			
Qwen3 (2025)	•	•	•	GSPO	•	•	
GLM-5 (2026)	•	•	•	seq. RL		•	•
MiniMax-M2.5 (2026)	•	•	process	CISPO		•	•

PT = Pretraining; SFT = Supervised Fine-Tuning; RM = Reward Model; RL = Reinforcement Learning; RS = Rejection Sampling; CoT-RL = Chain-of-Thought RL; Tool RL = Tool/Agent RL. OMD = Online Mirror Descent.

The trend is clear: the number of post-training stages has grown from zero (GPT-3) to one (InstructGPT’s SFT+RM+PPO counted as a single “RLHF” pipeline) to five or more sequential stages in frontier 2026 models. The most capable open-source models (GLM-5, MiniMax-M2.5, Qwen3) now use multi-phase RL pipelines where different RL stages target different capabilities (reasoning, agentic behavior, general alignment), connected by distillation to prevent forgetting. The specific RL algorithm used at each stage (PPO, GRPO, CISPO, DPO, etc.) has become less important than the overall pipeline architecture and the quality of reward signals.

Bibliography

- [1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep Reinforcement Learning from Human Preferences,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] L. Ouyang *et al.*, “Training Language Models to Follow Instructions with Human Feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [4] Z. Shao *et al.*, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [5] J. Hu, J. K. Liu, and W. Shen, “REINFORCE++: A Simple and Efficient Approach for Aligning Large Language Models,” *arXiv preprint arXiv:2501.03262*, 2025.
- [6] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model Is Secretly a Reward Model,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [7] M. G. Azar *et al.*, “A General Theoretical Paradigm to Understand Learning from Human Feedback,” *arXiv preprint arXiv:2310.12036*, 2024.
- [8] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “KTO: Model Alignment as Prospect Theoretic Optimization,” *arXiv preprint arXiv:2402.01306*, 2024.
- [9] J. Hong, N. Lee, and J. Thorne, “ORPO: Monolithic Preference Optimization without Reference Model,” *arXiv preprint arXiv:2403.07691*, 2024.
- [10] Y. Meng, M. Xia, and D. Chen, “SimPO: Simple Preference Optimization with a Reference-Free Reward,” *arXiv preprint arXiv:2405.14734*, 2024.
- [11] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [12] H. Dong *et al.*, “RAFT: Reward rAnked Fine-Tuning for Generative Foundation Model Alignment,” *Transactions on Machine Learning Research*, 2023.
- [13] H. Lee *et al.*, “RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback,” *arXiv preprint arXiv:2309.00267*, 2023.
- [14] Y. Bai *et al.*, “Constitutional AI: Harmlessness from AI Feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [15] N. Lambert *et al.*, “Tülu 3: Pushing Frontiers in Open Language Model Post-Training,” *arXiv preprint arXiv:2411.15124*, 2024.
- [16] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” *OpenAI Technical Report*, 2018.
- [17] Y. Zhu *et al.*, “Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books,” *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [18] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models Are Unsupervised Multitask Learners,” *OpenAI Technical Report*, 2019.

- [19] T. Brown *et al.*, “Language Models Are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [20] DeepSeek-AI, “DeepSeek-V3 Technical Report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [21] Meta AI, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [22] Meta AI, “The Llama 4 Herd of Models,” *Meta AI Blog*, 2025.
- [23] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi, “Cross-Task Generalization via Natural Language Crowdsourcing Instructions,” *Proceedings of the Association for Computational Linguistics (ACL)*, 2022.
- [24] J. Wei *et al.*, “Finetuned Language Models Are Zero-Shot Learners,” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [25] D. M. Ziegler *et al.*, “Fine-Tuning Language Models from Human Preferences,” *arXiv preprint arXiv:1909.08593*, 2019.
- [26] N. Stiennon *et al.*, “Learning to Summarize with Human Feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [27] Google DeepMind, “Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,” *arXiv preprint arXiv:2507.06261*, 2025.
- [28] D. Guo *et al.*, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [29] S. Kullback and R. A. Leibler, “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [30] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [31] Qwen Team, “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [32] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [33] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, “STaR: Bootstrapping Reasoning with Reasoning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [34] OpenAI, “OpenAI o1 System Card,” *arXiv preprint arXiv:2412.16720*, 2024.
- [35] Moonshot AI, “Kimi k1.5: Scaling Reinforcement Learning with LLMs,” *GitHub: MoonshotAI/Kimi-k1.5*, 2025.
- [36] R. Nakano *et al.*, “WebGPT: Browser-Assisted Question-Answering with Human Feedback,” *arXiv preprint arXiv:2112.09332*, 2021.
- [37] T. Schick *et al.*, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [38] Moonshot AI, “Kimi-Researcher: End-to-End RL Training for Emerging Agentic Capabilities,” *Moonshot AI Technical Report*, 2025.
- [39] Z.ai Team, “GLM-5: From Vibe Coding to Agentic Engineering,” *arXiv preprint*, 2025.
- [40] MiniMax Team, “MiniMax-M1: The World’s First Open-Weight Large-Scale Hybrid-Attention Reasoning Model,” *GitHub: MiniMax-AI/MiniMax-M1*, 2025.

- [41] MiniMax Team, “Forge: Scalable Agent RL Framework and Algorithm,” *MiniMax Technical Report*, 2025.